

INF01151 – Sistemas Operacionais II

Aula 05 - Semáforos

Prof. Alberto Egon Schaeffer Filho

Introdução

- Sistema computacional pode oferecer diversos mecanismos para sincronização, além das variáveis tipo *lock* (mutex)
 - Semáforos
 - Variáveis de condição
 - Monitores
- Porque outros mecanismos?
 - Facilitar a programação
 - Oferecer primitivas mais flexíveis e robustas
 - Exclusão mútua não é a única necessidade de programas concorrentes

Semáforos

- Semáforos definidos por E.W. Dijkstra (1968)
- Diversos possibilidades de usos:
 1. Proteger seção crítica de código através de exclusão mútua
 2. Alocação de recursos
 3. Sincronização de condição
 - Uma operação só pode executar após outra ter sido executada
- Conceito e nome motivados pela forma como tráfego em ferrovia é sincronizado para evitar colisões
 - Indica se o **trilho** a frente está **livre ou ocupado** por outro trem
 - Semáforos podem estar **acessos ou apagados**

Semântica e primitivas de semáforos

- É uma estrutura de dados que contém:
 - **Variável protegida** (contador com valor não-negativo) acessada/modificada por *primitivas atômicas próprias*
 - Lista de **fluxos de execução** a espera do semáforo
 - Importante: não é especificada a política da lista (fifo, lifo, prioridade, etc)
- Duas primitivas atômicas próprias
 - **P (Proberen, testar)** → Também chamado de **wait**
 - Se valor do semáforo for positivo, deve decrementá-lo e processo segue adiante; caso contrário, bloqueia processo até a ocorrência de um evento
 - **V (Verhogen, incrementar)** → Também chamado de **signal**
 - Sinaliza a ocorrência de um evento, incrementando valor do semáforo

Semáforos devem ser inicializados antes de serem utilizados:

Configura o valor da variável protegida (**conforme o uso**) e cria uma estrutura que armazena referências a processos em espera

Implementação de semáforos (original – Dijkstra)

```
Var semaphore s;  
  
....  
  
procedure P(s)  
begin  
    if (s > 0)  
        then s = s - 1;  
        else {bloqueia processo P em s};  
end;  
  
procedure V(s)  
begin  
    if (existe processos bloqueados em s)  
        then {desbloqueia um processo}  
        else s = s + 1;  
end;
```

- Considerações:
 - Variável semáforo possui valor não-negativo
 - As operações *P* e *V* são atômicas
 - Implementadas pelo sistema operacional (sem condição de corrida)
 - Disponibilizadas via uma API
 - Diferentes implementações
 - Espera ativa (*busy wait*)
 - Processos são bloqueados

P(s): < await (s > 0) s = s – 1; >

1º uso: exclusão mútua

```
Var semaphore s = 1;

Parbegin
  repeat
    P(s);
    {seção crítica}
    V(s);
    {seção não crítica}
  until (forever);
Parend.
End.
```

```
repeat
  P(s);
  {seção crítica}
  V(s);
  {seção não crítica}
until (forever);
```

init_semaphore(s, 1);

P(s);

Seção crítica

V(s);

- Tipo especial de semáforo: ***semáforo binário*** (ou *mutex*)
 - Assume apenas dois valores (1 e 0)
 - Indica se a seção crítica está livre (1) ou ocupada (0)
 - Se o semáforo estiver ocupado, processo é bloqueado

Inconvenientes de semáforos para executar exclusão mútua

- Não há garantias quanto à propriedade de *espera limitada*
 - Na semântica de semáforos não há nenhuma definição quanto a ordem que processos são desbloqueados pela primitiva V
 - Embora **algumas implementações** possam garantir uma determinada política, por exemplo, **fifo**
- A correção do programa depende do bom uso de P e V
 - Pode levar o programa a *deadlock*

2º uso: alocação de recursos

- Tipo especial de semáforo: ***semáforo contador***
 - Variável protegida é inicializada com um **valor ≥ 1**
 - Usado para representar um conjunto de recursos idênticos
- Princípio básico
 - ***Alocar recursos = $P(s)$***
 - Enquanto houver recursos, é possível usá-los sem ser bloqueado
 - Na falta de recursos, processo é bloqueado até que um seja liberado
 - ***Liberar recursos = $V(s)$***
 - Ao terminar de usar uma unidade do recurso, incrementa o contador
- Caso particular: valor = 1 (*recurso unitário* $\rightarrow$ *exclusão mútua*)
 - Semáforo binário (semáforo assume apenas 0 ou 1)

Alocação de recursos com semáforos contadores

```
Var semaphore s = 3; //exemplo
```

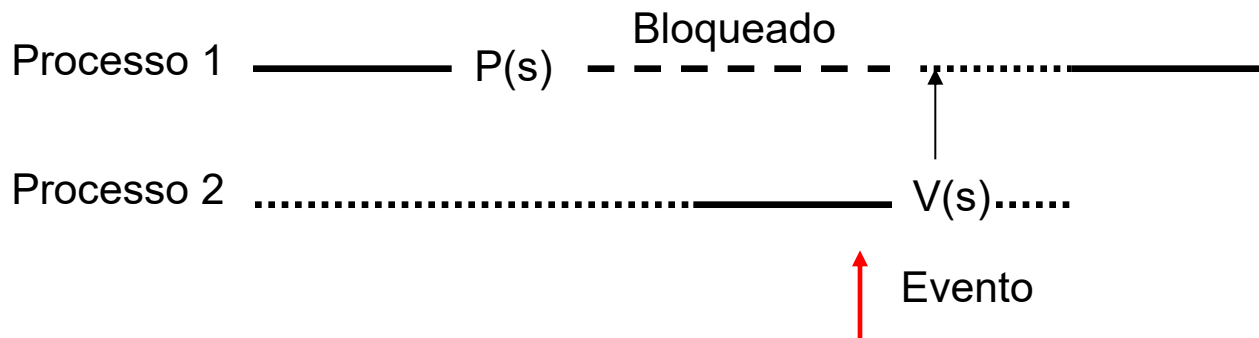
```
Parbegin
  repeat
    P(s);
    {acesso ao recurso}
    V(s);
    {restante do código}
  until (forever);
Parend.
End.
```

```
repeat
  P(s);
  {acesso ao recurso}
  V(s);
  {restante do código}
until (forever);
```

- Situação de uso:
 - Vários processos podem acessar recurso sem problema de consistência
- Emprego:
 - Inicializar uma variável semáforo com um valor c
 - n processos podem **usar recurso simultaneamente** ($1 \leq n \leq c$)

3º uso: sincronização de condição

- Notificar ou esperar a ocorrência de um evento particular
 - Primitiva **V** é também denominada de **signal**
 - Primitiva **P** é também denominada de **wait**
- Princípio de funcionamento:
 - Semáforo é inicializado com um **valor zero**
 - Para esperar o evento se faz um **P(s)**
 - Para sinalizar a ocorrência se faz um **V(s)**



Utilizando semáforos para sincronização de condição

- Situação de uso:
 - Quando um processo deve realizar uma **operação a_2** somente **após** um outro processo ter executado uma **operação a_1**
- Emprego:
 - Semáforo inicializado com **zero**
 - Um processo **espera** por um evento executando **$P(s)$**
 - Outro processo **sinaliza** um evento executando **$V(s)$**

```
Var semaphore s = 0
Parbegin
    ....
    P(s);
    executa operação  $a_2$ 
    ....
Parend.
End.
```

```
....
executa operação  $a_1$ 
V(s);
....
```

Resumo: uso de semáforos em programação concorrente

Uso	Descrição
Exclusão mútua	Obtido através de um semáforo inicializado em 1. Um processo realiza uma operação $P(s)$ antes de entrar na seção crítica e uma operação $V(s)$ no momento de sair. Tipo especial de semáforo (binário) também denominado de <i>mutex</i> .
Alocação de recursos	Permite o uso concorrente de um recurso por até n processos, ou seja, $1 \leq n \leq c$, onde c é uma constante. Um semáforo inicializado com c é usado para implementar a limitação de concorrência. São os semáforos denominados de <i>contadores</i> .
Sincronização de condição	Quando um processo necessita realizar uma operação a_2 somente após um outro processo ter realizado uma operação a_1 . Semáforo é inicializado em 0. Forma de sincronização também conhecida por <i>senalização</i> .

Exemplos de uso de semáforos: clássicos da programação concorrente

1. Jantar dos filósofos
2. Produtor-consumidor
 - a) Buffer unitário
 - b) Buffer limitado (com capacidade n)
3. Leitores e escritores

Jantar dos filósofos

- Considere um grupo de N filósofos que passam seu tempo **pensando** ou **comendo spaghetti**, mas obedecem algumas regras
 - Sentados em uma mesa circular
 - Existem apenas N garfos
 - São necessários dois garfos para comer
 - Filósofo só pode utilizar os garfos imediatamente de sua direita ou esquerda

```
Process Philosopher [int i=0 to N-1] {  
    while (true) {  
        Think  
        Acquire forks;  
        Eat;  
        Release forks;  
    }  
}
```



Objetivos

- Jantar dos filósofos ilustra problemas de concorrência
 - **Alocação de recursos:** garfos para filósofos
 - **Exclusão mútua:** acesso aos garfos
 - **Sincronização condicional:** só pode comer quando tiver os dois garfos
- Necessidade de evitar
 - Livelock
 - Deadlock
 - Starvation
- Uma solução: cada **garfo** é representado por um **semáforo**
 - Um filósofo tenta **adquirir um garfo** executando **P**
 - Um filósofo **libera um garfo** executando **V**

Primeira tentativa

```
N = 5
Var semaphore forks[N] = {1,1,1,1,1}

Process Philosopher [int i=0 to N-1] {
    int left = i;
    right = (i+1)%N;

    while (true) {
        Think;
        P(forks[left]);
        P(forks[right]);
        Eat;
        V(forks[left]);
        V(forks[right]);
    }
}
```

Problema?

Suponha que todos os filósofos fiquem com fome ao mesmo tempo, e que cada um consiga pegar seu garfo à esquerda. Quando um filósofo tentar pegar seu garfo à direita, ficará bloqueado para sempre (*deadlock*)

Deadlock

- Se todos os filósofos pegam o garfo da esquerda, ocorre um impasse entre eles (*deadlock*)
 - Deadlock pode ocorrer quando há uma espera circular de recursos (**mais tarde**: necessário 4 condições)
- Diversas soluções possíveis...
 - ① **Quebrar o ciclo**: um filósofo pega primeiro o garfo da direita, enquanto que os outros pegam primeiro o garfo da esquerda
 - ② **Controlar alocação de cadeiras**: permitir que no máximo $N-1$ filósofos sentem simultaneamente na mesa
 - ③ **Garantir aquisição de ambos os garfos**: permitir que um filósofo pegue os seus garfos somente se ambos estiverem disponíveis

Solução 1

- Um filósofo pega o garfo da direita e depois da esquerda, e os demais pegam primeiro da esquerda e depois o da direita

```
Process Philosopher [int i=0 to N-1] {  
    int left, right;  
  
    if (i==0) {  
        left = 1;  
        right = 0  
    }  
    else {  
        left = i;  
        right = (i+1)%N;  
    }  
    ...  
}
```

Solução 2

- Se no máximo $N-1$ filósofos tentarem comer haverá sempre um que terá a chance de pegar dois garfos
- Deve utilizar um semáforo adicional para representar o número máximo de filósofos autorizados a comer simultaneamente ($N-1$)

```
Var semaphore chairs =  $N-1$ ;  
...  
P(chairs) ;  
P(forks[left]);  
P(forks[right]);  
//Eat  
V(forks[left]);  
V(forks[right]);  
V(chairs) ;  
...
```

Produtor-consumidor

(de novo... mas agora com semáforos!!)

- É uma relação bastante comum entre processos
- Problema fundamental:
 - Como permitir diferentes taxas de produção e consumo sem haver problemas?
 - Não deve consumir o que não existe
 - Não deve produzir sobre o que já existe
- Variantes:
 - Buffer unitário, buffer limitado com capacidade n
 - 1 produtor e 1 consumidor / n produtores e m consumidores

Buffer com slot unitário

- Situação:
 - Múltiplos consumidores e produtores
 - Apenas um slot
- Duas razões para bloquear:
 - Produtores **esperam** que o *slot* esteja **vazio** para “produzir”
 - Consumidores **esperam** que o *slot* esteja **cheio** para “consumir”
- Solução emprega dois semáforos binários:
 - *empty* inicializado em 1
 - *full* inicializado em 0

```
Var semaphore empty = 1;  
    semaphore full   = 0;
```

```
Data buf; // buffer unitário
```

Buffer com slot unitário

```
Var semaphore empty = 1;  
    semaphore full = 0;
```

```
Data buf = null;
```

```
Process Producer[i=1 to M] {  
    while (true) {  
        //produce data  
        P(empty) ;  
        buf = data;  
        V(full) ;  
    }  
}
```

```
Process Consumer[j=1 to N] {  
    Data myData;  
    while (true) {  
        P(full) ;  
        myData = buf;  
        V(empty) ;  
        //Consume myData  
    }  
}
```

split binary semaphore: no máximo um entre *empty* ou *full* é 1, a qualquer momento (invariante)

Buffer limitado com capacidade n

- Uso de semáforo como contador de recursos
- Inicialização:
 - Empty – número de slots vazios = n
 - Full – número de slots cheios = 0
- Invariante:
 - $\text{empty} + \text{full} = n$

```
Var semaphore empty = n;  
    semaphore full  = 0;  
  
Data buf[n];  
int front = 0, rear = 0;
```

Buffer limitado com capacidade n: um único produtor e um único consumidor

```
Var semaphore empty = n;  
    semaphore full = 0;  
  
Data buf[n];  
int front = 0, rear = 0;
```

```
Process Producer {  
    while (true) {  
        //produce data  
        P(empty) ;  
        buf[rear] = data;  
        rear = (rear + 1) % n;  
        V(full) ;  
    }  
}
```

Exemplo: uso de semáforos para alocação de recursos

- Empty representa número de slots vazio
- Full representa número de slots cheios

```
Process Consumer {  
    while (true) {  
        //fetch data  
        P(full) ;  
        result = buf[front];  
        front = (front + 1) % n;  
        V(empty) ;  
    }  
}
```


Buffer limitado com capacidade n: múltiplos produtores

```
Var semaphore empty = n,  
    semaphore full = 0;  
    semaphore mutexP = 1, mutexC = 1;  
    int front = 0, rear = 0;
```

```
Process Producer [int i=1 to M] {  
    while (true) {  
        //produce data  
        P(empty);  
        P(mutexP);  
        buf[rear] = data;  
        rear = (rear+1)%n;  
        V(mutexP);  
        V(full);  
    }  
}
```

Com múltiplos produtores há disputa para usar e atualizar a variável *rear* (seção crítica)

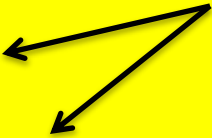
- Produtores e consumidores podem executar concorrentemente
- Mas dois produtores (ou dois consumidores) não

Buffer limitado com capacidade n: múltiplos consumidores

```
Var semaphore empty = n,  
    semaphore full = 0;  
    semaphore mutexP = 1, mutexC = 1;  
    int front = 0, rear = 0;
```

```
Process Consumer [int i=1 to N] {  
    Data myData;  
    while (true) {  
        //produce data  
        P(full);  
        P(mutexC);  
        myData = buf[front];  
        front = (front+1)%n;  
        V(mutexC);  
        V(empty);  
        //Consume myData;  
    }  
}
```

Com múltiplos consumidores
há disputa para usar e atualizar
a variável *front* (seção crítica)



Leitores e escritores

- Sistema com dados compartilhados onde um conjunto de processos apenas lê os dados (leitores) e outro conjunto modifica esses dados (escritores)
- Condições a serem satisfeitas
 - Os leitores podem ler concorrentemente
 - Os escritores precisam ter acesso exclusivo
 - Não deve ocorrer leitura se uma escrita estiver em curso, e vice-versa
 - Apenas um processo escritor pode alterar os dados em um dado instante de tempo
- Questão adicional: quem espera por quem?
 - Leitor tem uma prioridade não preemptiva sobre um escritor (*readers preferred readers-writes system*)
 - Escritor tem uma prioridade não preemptiva sobre um leitor (*writers preferred readers-writes system*)

Leitores e escritores:

readers preferred readers-writes system

```
Var semaphore mutex = 1;  
    semaphore rw = 1;  
    int readcount = 0;
```

```
Process Writer {  
    while (true) {  
        P(rw) ;  
        // writing is performed  
        V(rw) ;  
    }  
}
```

```
Process Reader {  
    while (true) {  
        P(mutex) ;  
        readcount++;  
        if (readcount == 1)  
            P(rw) ;  
        V(mutex) ;  
        // reading is performed  
        P(mutex) ;  
        readcount--;  
        if (readcount == 0)  
            V(rw) ;  
        V(mutex) ;  
    }  
}
```

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 4 (até seção 4.4.1 inclusive)
- Silberschatz, A. and Galvin, P. *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 6 (seção 6.5 e 6.6)

Exercício

- Barreira é um mecanismo de sincronização onde um grupo de tarefas se sincronizam em um determinado ponto do código. As 1^{as} tarefas a chegarem na barreira esperam pelas outras antes de prosseguirem. Quando a última tarefa do grupo chegar à barreira, todas poderão prosseguir.
 - Implemente em pseudo-código uma barreira para dois processos utilizando semáforos